Package 'geomattR'

January 11, 2026

Type Package

Title Calculate Geometric Attributes of Spatial Polygons

Version 0.1.0

Description Provides a comprehensive toolkit for calculating geometric and morphometric attributes of spatial polygons. Computes metrics including area, perimeter, compactness, elongation, orientation (bearing), fractal dimension, and various shape indices. All measurements use geodesic calculations for accuracy across different coordinate reference systems, automatically handling geographic (lon/lat) data appropriately. Designed for geospatial analysts, urban planners, and environmental scientists working with 'terra' vector data objects <https://github.com/rspatial/terra>.

License MIT + file LICENSE

URL https://github.com/gortegasolis/geomattR

BugReports https://github.com/gortegasolis/geomattR/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Imports terra (>= 1.8.70), geosphere (>= 1.5.20), methods

Suggests testthat (>= 3.2.3), knitr (>= 1.50), rmarkdown (>= 2.30)

Config/testthat/edition 3

VignetteBuilder knitr

NeedsCompilation no

Author Gabriel Ortega-Solis [aut, cre] (ORCID: <https://orcid.org/0000-0002-0516-5694>)

Maintainer Gabriel Ortega-Solis <g.ortega.solis@gmail.com>

Contents

`geomattR-package` *geomattR: Calculate Geometric Attributes of Spatial Polygons*

Description

Calculate geometric and morphometric attributes of spatial polygons with geodesic accuracy. Computes area, perimeter, compactness, elongation, orientation, fractal dimension, and shape indices suitable for geospatial analysis, urban planning, and environmental science applications.

All measurements use geodesic calculations for accuracy across different coordinate reference systems. The package automatically handles both geographic (lon/lat) and projected CRS appropriately.

Details

Main Functions:

- `calculate_geometric_attributes()`: Calculate metrics for one or more polygons (sequential)
- `calculate_geometric_attributes_single()`: Core computation for a single polygon
- `calculate_geometric_attributes_parallel()`: Calculate metrics with parallel processing
- `get_distant_points()`: Find most distant points on convex hull
- `calc_elongation()`: Calculate elongation ratio
- `calc_extent_ew()`: Calculate east-west extent
- `calc_extent_ns()`: Calculate north-south extent

Geodesic Measurements:

All calculations prioritize accuracy:

- **Area & Perimeter**: Use `terra::expanse()` with explicit `transform = TRUE` for automatic geodesic calculation; perimeter is projected to lon/lat as recommended by terra documentation
- **Distances**: Use explicit `method = "geo"` for geodesic calculations
- **Automatic Projection**: Non-geographic CRS are handled transparently

Example:

```
library(geomattR)
library(terra)

# Create sample polygon in WGS84
coords <- cbind(c(0, 0, 1, 1, 0), c(0, 1, 1, 0, 0))
polygon <- vect(coords, type = "polygon", crs = "EPSG:4326")

# Calculate all attributes (geodesic by default)
result <- calculate_geometric_attributes(polygon)

# Calculate specific metrics
result <- calculate_geometric_attributes(polygon,
  metrics = c("area", "perimeter", "compactness"))
```

Author(s)

Maintainer: Gabriel Ortega-Solis <g.ortega.solis@gmail.com> (ORCID)

See Also

Useful links:

- https://github.com/gortegasolis/geomattR
- Report bugs at https://github.com/gortegasolis/geomattR/issues

---

`calculate_geometric_attributes` *Calculate Geometric Attributes of Spatial Polygons*

Description

Calculates a comprehensive set of geometric and morphometric attributes for spatial polygon features including area, perimeter, compactness metrics, shape indices, elongation, and orientation measures. If multiple features are provided, processes each feature sequentially.

Usage

```
calculate_geometric_attributes(v, metrics = "all")
```

Arguments

`v`	A SpatVector object representing one or more polygons.
`metrics`	Character string or vector specifying which metrics to calculate. Options are: • `"all"` (default): Calculate all available metrics • A single metric name as a string: Calculate only that metric • A character vector of metric names: Calculate specified metrics

Available metric names: "area", "perimeter", "compactness", "reock", "elongation_rectangle", "num_holes", "hole_area", "hole_area_pct", "num_polygons", "ew_length", "ns_length", "maxlength", "bearing", "northerness", "fractaldimension", "sinuosity", "shape_index", "circularity_ratio", "decimallongitude", "decimallatitude"

Details

The function calculates various geometric metrics that describe polygon shape, size, and orientation. For multiple features, each polygon is processed sequentially. For parallel processing, use `calculate_geometric_attributes_parallel`.

Size metrics:

- Area calls directly to terra::expanse() for geodesic area calculation.
- Perimeter calls directly to terra::perimeter().
- Extents (EW, NS) are based on the geographic bounding box of each supplied polygon. The extracts the north-west, north-east, south-west, and south-east corners and calculates the average east-west and north-south distances using geodesic methods. **Shape metrics:**
- Compactness measures how closely a polygon resembles a circle
- Shape index and circularity provide alternative shape characterizations
- Fractal dimension measures boundary complexity

Orientation metrics:

- Bearing describes the orientation of the longest axis of the polygon
- Northerness quantifies the northward component of the bearing. Values range from -1 (southward) to 1 (northward).
- Elongation takes the ratio between the major and minor axes of the minimum bounding rectangle.

Value

The input SpatVector with additional columns containing the requested geometric attributes:

- `area`: Total area in square meters
- `perimeter`: Total perimeter length in meters
- `compactness`: Polsby-Popper compactness (1 = perfect circle)
- `reock`: Reock compactness (area / minimum enclosing circle area)
- `elongation_rectangle`: Elongation ratio based on minimum bounding rectangle
- `num_holes`: Number of holes (interior rings) in the polygon
- `hole_area`: Total area of holes in square meters
- `hole_area_pct`: Percentage of total area occupied by holes
- `num_polygons`: Number of separate polygon parts (multi-part count)
- `ew_length`: Average east-west extent in meters
- `ns_length`: Average north-south extent in meters

- maxlength: Maximum distance across convex hull in meters
- bearing: Geographic bearing of maximum length line in degrees
- northerness: Northerness component of bearing (-1 to 1)
- fractaldimension: Fractal dimension (complexity measure, typically 1-2)
- sinuosity: Sinuosity (perimeter to maximum length ratio)
- shape_index: Shape index (deviation from circular shape)
- circularity_ratio: Circularity based on maximum length
- decimallongitude: Centroid longitude in decimal degrees
- decimallatitude: Centroid latitude in decimal degrees

Examples

```
## Not run:
library(terra)

# Load a polygon shapefile (single or multiple features)
polygons <- vect("path/to/polygons.shp")

# Calculate all geometric attributes (processes sequentially)
polygons_with_attrs <- calculate_geometric_attributes(polygons)

# Calculate specific metrics only
polygons_subset <- calculate_geometric_attributes(polygons,
                     metrics = c("area", "perimeter", "compactness"))

# View the results
print(polygons_with_attrs)

## End(Not run)
```

---

calculate_geometric_attributes_parallel

Calculate Geometric Attributes in Parallel

Description

Calculates geometric attributes for multiple polygons. If a cluster is provided, processes polygons in parallel; otherwise defaults to sequential processing.

Usage

```
calculate_geometric_attributes_parallel(v, metrics = "all", cl = NULL)
```

Arguments

v	A SpatVector object representing one or more polygons.
metrics	Character string or vector specifying which metrics to calculate. See calculate_geometric_attributes for available options.
cl	A cluster object created by parallel::makeCluster(), or NULL (default). If NULL, the function will process polygons sequentially. To enable parallel processing, pass an explicit cluster object.

Details

This function processes each polygon independently using the internal single-polygon function. If a cluster is provided via the cl parameter, it will process polygons in parallel. Otherwise, it defaults to sequential processing (equivalent to calling calculate_geometric_attributes).

For a single polygon, this function simply calls calculate_geometric_attributes directly.

To use parallel processing, create a cluster first:

```
cl <- parallel::makeCluster(parallel::detectCores() - 1)
result <- calculate_geometric_attributes_parallel(polygons, cl = cl)
parallel::stopCluster(cl)
```

Value

A SpatVector object with the same structure as input, with added columns containing the requested geometric attributes.

Examples

```
## Not run:
library(terra)

# Load multiple polygons
polygons <- vect("path/to/polygons.shp")

# Process sequentially (default)
result <- calculate_geometric_attributes_parallel(polygons)

# Process in parallel with explicit cluster
cl <- parallel::makeCluster(parallel::detectCores() - 1)
result <- calculate_geometric_attributes_parallel(polygons, cl = cl)
parallel::stopCluster(cl)

## End(Not run)
```

`calculate_geometric_attributes_single` *Calculate Geometric Attributes for a Single Polygon*

Description

Calculates geometric attributes for exactly one polygon feature. This is the core computational function used by `calculate_geometric_attributes` and `calculate_geometric_attributes_parallel`.

Usage

```
calculate_geometric_attributes_single(v, metrics = "all")
```

Arguments

`v`	A SpatVector object containing exactly one polygon feature.
`metrics`	Character string or vector specifying which metrics to calculate. Options are: • `"all"` (default): Calculate all available metrics • A single metric name as a string: Calculate only that metric • A character vector of metric names: Calculate specified metrics Available metric names: "area", "perimeter", "compactness", "reock", "elongation_rectangle", "num_holes", "hole_area", "hole_area_pct", "num_polygons", "ew_length", "ns_length", "maxlength", "bearing", "northerness", "fractaldimension", "sinuosity", "shape_index", "circularity_ratio", "decimallongitude", "decimallatitude" (see Details for descriptions).

Details

This function is optimized to process a single polygon and only computes intermediate geometric objects (convex hull, centroid, etc.) that are needed for the requested metrics.

Geodesic Measurements: All measurements use geodesic calculations for accuracy across different coordinate systems. Area calculations use `terra::expanse()` with automatic lon/lat transformation. Perimeter calculations explicitly project to lon/lat (EPSG:4326) if the input is not already in a geographic CRS, ensuring accurate geodesic measurements as recommended by terra documentation. Distance-based metrics (maxlength, ew_length, ns_length, bearing) use explicit geodesic distance calculations via `method = "geo"`.

Area returns the total area in square meters using geodesic methods. Perimeter returns the total perimeter length in meters using geodesic methods. Compactness is calculated using the Polsby-Popper formula: (4 * pi * Area) / (Perimeter^2). The formula produces values between 0 and 1, where 1 indicates a perfect circle. Reock compactness is calculated as the ratio of the polygon's area to the area of its minimum enclosing circle. Elongation ratio is calculated as the ratio of the polygon's length to its width based on the minimum bounding rectangle. Number of holes counts the number of interior holes within the polygon. Hole area returns the total area of all interior holes in square meters using geodesic methods. Hole area percentage calculates the percentage of the polygon's total area that is occupied by holes. Number of polygons counts the number of separate

polygon parts in a multi-part polygon. East-West length returns the average east-west extent of the polygon in meters based on the geographic bounding box aligned with the cardinal directions, using geodesic distance calculations. North-South length returns the average north-south extent of the polygon in meters based on the geographic bounding box aligned with the cardinal directions, using geodesic distance calculations. Maximum length returns the maximum distance in meters between a pair of opposite vertices across the polygon's convex hull, using geodesic calculations. Bearing returns the geographic bearing in degrees from the southernmost to the northernmost point of the polygon's convex hull. Northerness calculates the cosine of the bearing angle (in radians) to quantify the northward orientation of the polygon. Fractal dimension is calculated using the formula: 2 * (log(Perimeter) / log(Area)), providing a measure of shape complexity. Sinuosity is calculated as the ratio of the polygon's perimeter to its maximum length. Shape index is calculated as: Perimeter / (2 * sqrt(pi * Area)), providing a dimensionless measure of shape complexity. Circularity ratio is calculated as: (4 * Area) / (pi * Maximum Length^2), indicating how closely the shape resembles a circle. Decimal longitude returns the centroid's longitude in decimal degrees. Decimal latitude returns the centroid's latitude in decimal degrees.

For processing multiple polygons, use `calculate_geometric_attributes` (sequential) or `calculate_geometric_attribu` (parallel).

Value

The input SpatVector with additional columns containing the requested geometric attributes.

Examples

```
## Not run:
library(terra)

# Load a single polygon
polygon <- vect("path/to/polygon.shp")[1, ]

# Calculate all attributes
result <- calculate_geometric_attributes_single(polygon)

# Calculate only specific metrics
result <- calculate_geometric_attributes_single(polygon,
  metrics = c("area", "perimeter", "compactness"))

## End(Not run)
```

---

calc_elongation *Calculate Elongation Ratio from Minimum Bounding Rectangle*

Description

Calculates the elongation ratio of a polygon based on its minimum bounding rectangle. The ratio is the major axis length divided by the minor axis length.

Usage

```
calc_elongation(v)
```

Arguments

v	A SpatVector object representing a polygon.

Value

A numeric value representing the elongation ratio (major/minor axis). Higher values indicate more elongated shapes.

Examples

```
## Not run:
library(terra)
polygon <- vect("path/to/polygon.shp")
elongation <- calc_elongation(polygon)

## End(Not run)
```

---

calc_extent_ew *Calculate East-West Extent*

Description

Calculates the average east-west extent of a polygon based on its geographic bounding box.

Usage

```
calc_extent_ew(v)
```

Arguments

v	A SpatVector object representing a polygon.

Value

A numeric value representing the average east-west extent in meters.

Examples

```
## Not run:
library(terra)
polygon <- vect("path/to/polygon.shp")
ew_extent <- calc_extent_ew(polygon)

## End(Not run)
```

calc_extent_ns *Calculate North-South Extent*

Description

Calculates the average north-south extent of a polygon based on its geographic bounding box.

Usage

```
calc_extent_ns(v)
```

Arguments

v	A SpatVector object representing a polygon.

Value

A numeric value representing the average north-south extent in meters.

Examples

```
## Not run:
library(terra)
polygon <- vect("path/to/polygon.shp")
ns_extent <- calc_extent_ns(polygon)

## End(Not run)
```

get_distant_points *Find largest distance between a pair of opposite vertices*

Description

Identifies the pair of most distant points and calculate the distance and bearing from the southernmost to the northernmost point.

Usage

```
get_distant_points(v, distance = TRUE, bearing = TRUE)
```

Arguments

v	A SpatVector object representing a polygon.
distance	Logical indicating whether to return the maximum distance (default TRUE).
bearing	Logical indicating whether to return the bearing (default TRUE).

Value

A list containing:

south_point	SpatVector of the southernmost point
north_point	SpatVector of the northernmost point
distance	Maximum distance in meters
bearing	Geographic bearing from south to north in degrees

Examples

```
## Not run:
library(terra)
polygon <- vect("path/to/polygon.shp")
distant_pts <- get_distant_points(polygon)

## End(Not run)
```

Index